Affine Kernels Have Closed-Form Memory Behavior

What an automatic symbolic analysis derives about scientific and ML kernels, and what you can do with the formulas.

1. The cost that arithmetic doesn't count

The kernels at the heart of scientific computing and machine learning — matrix multiplication, convolutions, stencils, the building blocks of attention — are *affine loop nests*: loops whose array subscripts are linear in the loop counters. For such kernels, arithmetic cost is a solved problem. You count: matrix multiplication of two n x n matrices does $2n^3$ operations. Double n, expect 8x the work. This kind of scaling statement is exact, symbolic, and free.

Run the kernel, though, and the prediction fails in a specific way. Here are two versions of matrix multiplication with *identical* arithmetic — the same $2n^3$ operations, the same $3n^3$ array accesses, only the order of the two inner loops swapped:

```
for (i) for (j) for (k)          for (i) for (k) for (j)
    C[i][j] += A[i][k]*B[k][j];      C[i][j] += A[i][k]*B[k][j];
```

On real hardware the right one runs several times faster. Arithmetic cannot explain this; nothing about "$2n^3$" changed. The missing cost is *data movement*. A processor computes only on data held in a small fast memory next to the core — a *cache*, typically 32 KB close in, a few MB further out — and fetching a value that is not there from main memory costs tens to hundreds of times more than the multiply it feeds. A kernel's real cost is arithmetic plus traffic, and traffic depends on something arithmetic counting never sees: *whether the data you touch is still in the cache from the last time you touched it*.

Scaling behavior of arithmetic: symbolic, exact, effortless. Scaling behavior of memory traffic: until recently, you measured it — run the kernel (or a cache simulator) at one problem size on one machine, get one number, repeat for every size, machine, and program variant you care about. This document is about the discovery that for affine kernels the memory side can be made just as symbolic as the arithmetic side — by a compiler, automatically — and about how much falls out of that.

2. The one quantity that decides everything

When does a memory access cost nothing, and when does it cost a trip to main memory? Caches keep the most recently used data. So the question for each access is: *since the last time this value was touched, how much other data has been touched?* If that in-between amount fits in the cache, the value is still there — the access is free. If not, it was evicted — the access is a *miss*.

Call the in-between amount the **reuse distance** of the access: the number of distinct values touched since its previous use. This single notion compresses everything relevant about a cache of any size C:

> an access misses exactly when its reuse distance exceeds C.

So if you know, for a given kernel, *how many accesses have which reuse distance*, you know the fraction of accesses that miss — the traffic — for **every** cache size at once. Not a measurement of one configuration: the whole curve.

3. Matrix multiplication by hand

The remarkable thing about affine kernels is that this histogram of reuse distances is not an unstructured mess. Walk through the triple loop `for i, for j, for k: C[i][j] += A[i][k]`

* `B[k][j]` and ask, for each of the three arrays, when a value is touched again:

  - **C[i][j]** is touched again on the very next k-iteration. In between: one element of A, one of B. Reuse distance ~3. This happens on essentially every iteration — one third of all accesses.
  - **A[i][k]** is touched again one j-iteration later. In between: one row of B-column-slices and the C entries — about 2n values. Another third of all accesses, at distance ~2n.
  - **B[k][j]** is touched again only when i advances — after a full sweep of one A-row per j, one C-row, and B itself: about n^2 values. The last third, at distance ~n^2.

Three loops, three kinds of reuse, three distances: 3, 2n, n^2. The miss fraction as a function of cache size C is therefore a *staircase*:

if the cache holds …	then the miss fraction is …
less than 3 values	1 (everything misses)
3 … 2n values	2/3
2n … n^2 values	1/3
more than ~n^2 (one matrix)	~2/(3n), nearly zero

An execution of n^3 steps — billions of accesses — collapses into four lines of formulas. Each step of the staircase has a physical meaning: the running sum fits; then a row fits; then a whole matrix fits. And each machine question becomes a lookup: a cache of C values sits on one of the steps, and the step height is the traffic.

One loose end: the *first* touch of each value has no previous use — no distance to assign — and gets awkward in symbolic form. The paper this repository builds on resolves it with a move that is as practical as it is convenient: analyze the kernel *as if it runs repeatedly*, so every first touch is a re-touch from the previous round (its "imaginary reuse"). That is not a modeling fiction — it is how these kernels are actually used: time steps, solver sweeps, training iterations. (For a genuinely one-shot run, the repetition is easy to subtract back out.) For matmul, the imagined repetition gives the leftover 2/(3n) of accesses a distance of ~$3n^2$ — all the data — which is exactly the "cache holds everything" step of the staircase.

4. The step that makes it automatic

Hand-deriving that table for matmul takes a paragraph. For a 6-deep tiled loop nest, or a kernel with triangular bounds, it is hopeless. The result this repository studies is a compiler analysis (built on the "algebraic locality" paper) that derives the table *automatically and symbolically* for any affine kernel: because subscripts are linear in loop counters, "how many accesses have reuse distance D(n)" is a question about counting integer points in parametric polyhedra, which can be answered in closed form. Machine time: on the order of a minute per kernel, once. Output: a table of pairs

```
(reuse distance as a formula of the loop bounds,
 number of accesses at that distance, as a formula)
```

For the naive matmul above, the tool's exact table is the hand analysis with the constants filled in: distances 3, 2n+2-1/n, n^2+3n-1/n, ~$3n^2$, each carrying close to one third, one third, one third, and 2/(3n) of the accesses. The hand-waving is gone; the fractions are exact, down to the 1/n terms.

Two practical notes, then the payoff. First, hardware moves data in 64-byte *lines* of eight values, which shifts the constants (the same analysis at line granularity puts naive matmul's big step at 37% rather than 1/3); all machine-level numbers below are line-granular. Second, everything below is computed from the tables of 22 PolyBench kernels plus a family of matmul variants, derived fresh by the tool; nothing is measured on hardware. The paper itself

validated the tables against cycle-level cache simulation (about 1% average miss-ratio error), so the interesting question is not whether the formulas are right — it is what they are *for*.

5. What the formulas answer

5.1 Every cache size and every problem size, from one derivation

The table *is* the miss-ratio curve, for all cache sizes and problem sizes simultaneously. Evaluating it at a machine point costs microseconds: matmul at n = 2048 on a 32 KB cache — 37% of accesses miss; on 1 MB — still 37%; the number becomes small only past ~32 MB. A simulation sweep to produce the same curve runs the kernel once per cache size; here it is one substitution per question. That alone replaces a profiling campaign, but it is the *least* interesting consequence.

5.2 The cliffs have closed forms

Because the staircase steps are formulas, so are the problem sizes at which a given machine falls off them. Naive matmul's last step is "one matrix fits": n^2 values x 8 bytes <= C. So the largest safe problem is

```
n*(C) = sqrt(C_bytes / 8):    64 for 32 KB,  362 for 1 MB,
                              2048 for 32 MB.
```

Below n*, the kernel is gentle on the memory system no matter what; the first n past n*, traffic jumps by orders of magnitude. Every kernel has such a schedule of cliff sizes, and the analysis hands it to you in closed form. This is scaling behavior in the practical sense: not "the exponent is 3", but *at which n, on your machine, the behavior changes regime, and what it costs when it does*.

5.3 "Which version should I run?" gets an honest answer

The two matmuls from Section 1: the formulas say the k-inner version misses 37% of accesses and the j-inner version 8.3% — at *every* cache size up to 512 KB, for every large n. A 4.5x traffic difference between programs that every asymptotic method (including our own earlier data-movement-complexity study on this suite) scores as *identical*, because no exponent changes. Conversely, a single hardware measurement would reveal the 4.5x at one point but not that it is flat in C and n — that it is a property of the program pair, not of the machine.

Tiling — rewriting matmul to work on b x b blocks — is where the formula view pays most. Deriving the tables for tiled variants and dividing:

traffic reduction vs. naive	4 KB	8 KB	32 KB	512 KB	16 MB	64 MB
tile 8	36x	36x	36x	8x	8x	1.0x
tile 16	48x	50x	72x	8x	16x	1.0x
tile 32	8.5x	8.5x	**106x**	16x	31x	1.0x

(n = 2048.) Three honest answers to "should I tile?", all visible at once and none available from a yes/no analysis:

- *How much it pays depends on the cache*: 36x to 106x at realistic sizes — and exactly 1.0x once one matrix fits (the 64 MB column; or equivalently once n <= n*(C)). "Tile this kernel" is not a property of the kernel; "tile it when n > sqrt(C/8 bytes)" is.

- *The best tile size is a cache-size decision*: tile-32's working set is 24 KB, so at 8 KB it is four times *worse* than tile-8. The crossover points are the tiled kernels' own step boundaries.
- *What tiling actually does* is now precise: it does not change the cliff schedule (all variants share the 64 MB column); it lowers the step heights between the cliffs, by roughly the tile size.

5.4 A whole benchmark suite on one page — and a warning

Each of the 22 PolyBench kernels reduces to a handful of staircase rows. Reading them side by side (all tables in `tables/`):

- **gemm** (the BLAS-order matmul) misses 3.1% at 32 KB — and still 3.1% at 1 MB, and at 16 MB: one long step means *three levels of a real cache hierarchy are equivalent for this kernel*, a fact worth knowing before buying hardware or blaming L2.
- **2mm/3mm** (chained matmuls) sit at 28% until a 142 KB threshold (= 9n/8 lines, two matrix rows of the chain), then drop to 3.1%.
- **trisolve** has *no* step between "a few rows" and "the whole matrix": its 3.1% miss floor cannot be improved by any tiling until all data fits. The staircase proves a negative: nothing to gain.
- **trmm** was classified by our earlier asymptotic study as "already local, no headroom" — the same class as trisolve. The formulas disagree: trmm misses 50% at 32 KB and 3.1% at 1 MB. Sixteen-fold waste at L1, invisible to the exponent view that lumped the two kernels together, obvious in the staircase.

The warning generalizes. It is tempting to compress each kernel's table into a single "total data movement" score and rank kernels by it. The compression is provably treacherous: the table obeys a sum rule (reuse distances times their frequencies add up to the data size), so any score that weights long distances is dominated by rows carrying *vanishing* fractions of the accesses — precisely the rows that real caches never notice. On this suite the score misranks concretely: gesummv scores *worse* than mvt (33.8 vs 29.4) yet misses 3.3x *less* at 32 KB. Kernels within 5% of each other's scores differ up to 6x in actual traffic. The staircase is small — keep all of it; the constants and thresholds, which any single number discards, are where the machine-level truth lives.

5.5 Parallelism, by substituting into the same formulas

The derived formulas are symbolic in *each loop bound separately* — not just in one size n. That has a consequence that would be easy to miss: giving each of p cores a slice of one loop produces, per core, *the same kernel with that bound divided by p*. Parallel memory behavior needs no new theory and no new analysis run — only substitution into formulas that already exist. Doing so for matmul (n = 2016, per-core caches):

- **Split by rows of the output** (slice the i-loop): total traffic is *provably independent of p* — 1.02e9 lines whether p is 1 or 1008, at 32 KB and at 1 MB per core. The reason is visible in the algebra: the re-swept matrix B appears whole in every worker's formula; row parallelism never shrinks anyone's working set. A thousand private caches buy zero traffic relief.
- **Split by columns** (slice j): the per-worker working set *does* carry the 1/p, and the formulas predict that p one-MB caches begin to act as one big cache at $p = n^2 \cdot 8B/1MB \approx 32$. Substituting: at p = 32 exactly, total traffic collapses 61x (1.02e9 → 1.7e7 lines). The same substitution at 32 KB shows the collapse *never* comes — a sliced matrix still spans one 64-byte line per row, n lines = 126 KB minimum, so no column count fits it into 32 KB; and slices narrower than the 8-value line *raise* traffic again (visible as the p

= 1008 row worsening). Cache-line granularity, the classic silent killer of naive parallel reasoning, falls out of the formulas unasked.

Same kernel, same arithmetic, same degree of parallelism — and the two decompositions differ by 61x in memory traffic, with the threshold p = 32 and the 126 KB floor both computed before running anything.

5.6 Formulas that check themselves

A last property with practical weight: the tables carry internal consistency laws — the access fractions must sum to one, and the sum-rule above must hit the data size exactly. These are not decorative. Run mechanically over the suite, they *caught* six kernels (the time-stepped stencils, e.g. jacobi, heat-3d) whose analyzer output violates conservation by up to 54%; those are excluded above and reported rather than silently plotted. They also localized a smaller defect: gemm's table is short exactly one matrix worth of long-distance mass — the checks say not only *that* something is missing but *what and where it matters* (only the final step). A simulator that double-counted 13% of a trace would hand you a plausible-looking curve; a broken formula cannot hide from its own conservation law. (One honest caveat survives the checks: for triangular kernels — cholesky, lu, syrk — the tool's current approximation under-resolves the longest distances; their staircases look too flat, and we withhold claims about them until an exact mode lands.)

6. What this adds up to

Affine kernels — the loops scientific computing and machine learning are made of — turn out to admit *closed-form memory behavior*: a compiler can reduce an n^3-step execution to a four-row table of formulas, automatically, in about a minute, and the table answers in microseconds what previously required simulation sweeps per machine, per size, per variant:

- the miss ratio at any cache size and problem size;
- the problem sizes where behavior falls off a cliff, per machine, in closed form (sqrt(C/8) for matmul);
- which program version wins, by how much, and *under what conditions* — loop order (4.5x, unconditionally), tiling (36–106x, exactly when one matrix doesn't fit, with the tile size a formula of the cache);
- which kernels have improvement headroom at which cache level, and which provably have none — including cases where asymptotic classification gets the answer backwards;
- how memory traffic responds to parallel decomposition, including which splits are provably useless, the core count where private caches merge into one, and line-granularity floors — all by substituting p into formulas derived once;
- and whether to trust any of it, via conservation laws the tables must satisfy.

Arithmetic scaling has been symbolic since we learned to count operations. The memory side — the side that actually limits these kernels on real machines — was measurement-only. For affine programs it no longer is: the program *is* the model, the compiler extracts it, and locality becomes something you solve, not something you sample.

Appendix: provenance and reproduction

All numbers derive from the AutoLALA analyzer (`dmd-cli`) under the paper's canonical configuration — infinite repeat, scale approximation in Barvinok, 64-byte lines of eight f64 values — over the extracted PolyBench DSLs in `dsl/` plus matmul variants (naive ijk/ikj, tiles 8/16/32) written for this study; the naive-matmul table also reproduces the paper's element-granularity

Table 1 exactly (`tables/anchors.md`). Suite evaluations use n = 2016 (2048 for the matmul family); cache sizes are counted in lines (32 KB = 512 lines).

Pipeline: `run_suite.py` (analyzer runs, ~10 min) → `regimes.py` (exact symbolic extraction: piecewise quasi-polynomial parsing, exact polynomial fits with held-out verification, scale clustering; Fraction arithmetic throughout) → `derived.py`, `parallel_study.py`, `anchor_checks.py` (all tables cited above, under `tables/`: `machine_map.md`, `tiling.md`, `parallel.md`, `signatures.md`, `dmd_inversion.md`, `suite_regimes.md`, `anchors.md`). Conservation excluded heat-3d (1.54), jacobi-1d (1.33), jacobi-2d (1.27), seidel-2d (0.93), fdtd (0.96), imperfect (0.97); convolution is analyzed as a 9x9-filter variant (its image and filter are genuinely different scales and cannot both be bound to n).

```
python3 run_suite.py && python3 regimes.py && python3 derived.py
python3 parallel_study.py && python3 anchor_checks.py
pandoc REPORT.md -o REPORT.pdf --pdf-engine=xelatex \
  -V mainfont="DejaVu Serif" -V monofont="DejaVu Sans Mono" \
  -V geometry:margin=1in
```